

TDD

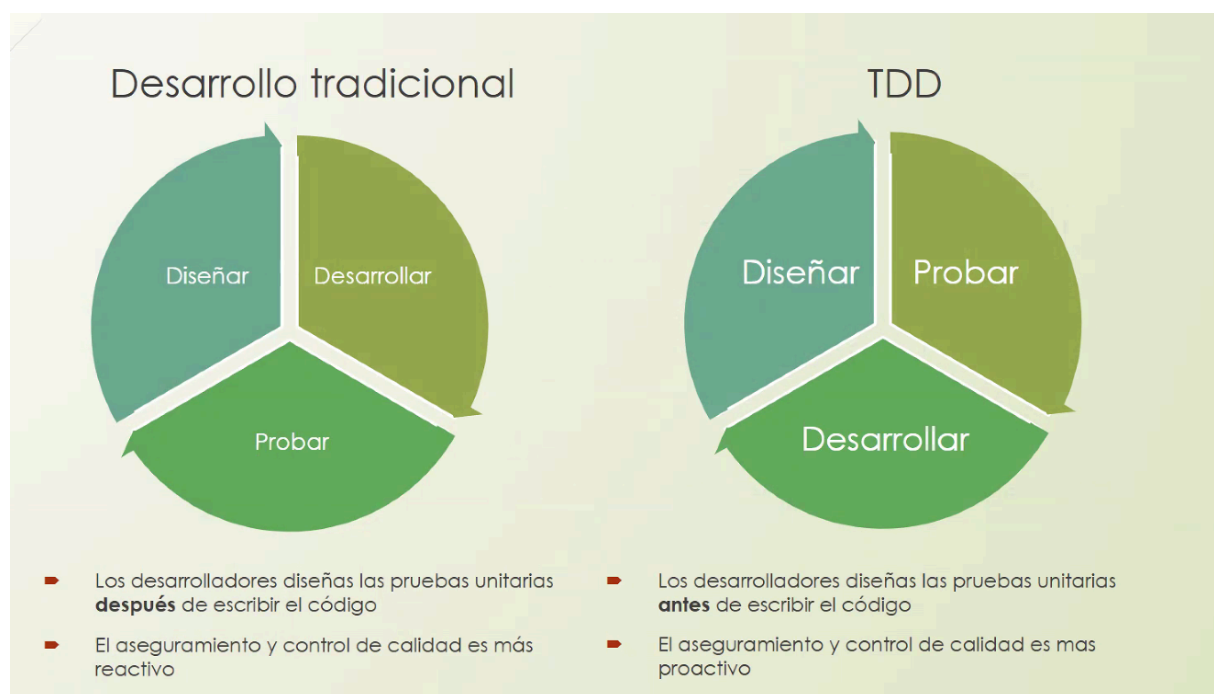
Unidad

Unidad 2

Cambia el pensamiento de la gestión tradicional y plantea que primero se escriban las pruebas y luego va el desarrollo.

El proceso mental que debe desarrollarse para crear test útiles puede descubrir y eliminar problemas en todas las etapas del desarrollo.

Desarrollo tradicional vs TDD



El TDD es proactivo busco diseñar esas pruebas antes de escribir el código y luego el código los voy a ir escribiendo en base a las pruebas que ya hice.

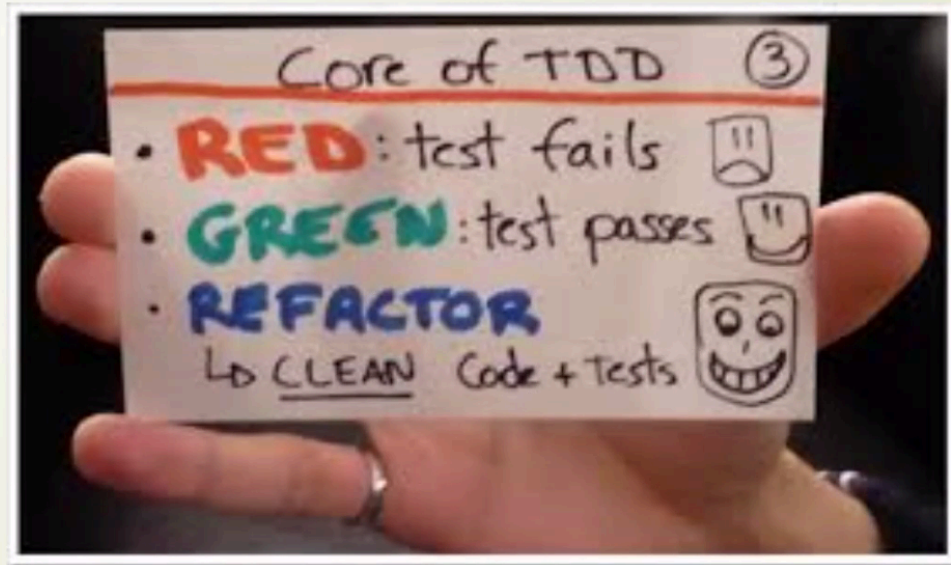
Que tiene que hacer el sw antes de decidir como lo vamos a hacer.

TDD → Desarrollo guiado por pruebas de software. Involucra escribir las pruebas primero por lo general son las pruebas unitarias.

Así sabemos cuando el código esta terminado, mas robusto, mas tolerable al cambio.

El Corazón de TDD

El ciclo Red-Green-Refactor



Primero escribimos un test buscamos que falle. Luego hacemos un cambio para que el código pase y luego hacemos re-factor es decir que escribimos un código mejorado.

3 LEYES DEL TDD

Siempre buscamos pruebas unitarias la idea es que sea automatizada

- No escribir una línea de código hasta que no hayas escrito antes un test case de falla
- No es necesario escribir mas de un test para que falle.
- No escribir mas código en producción que el necesario para que el test pase

Patrones de TDD

Como vamos a escribir los test antes de empezar

Patrones *red bar*

- One Step Test: Escribe un solo test que verifique una pieza mínima de funcionalidad.
- Starter Test: Primer test que te ayuda a explorar la API o comportamiento deseado.
- Explanation test: Escribe comentarios o preguntas para aclarar qué esperas que haga el código
- Learning test: permite conocer APIs externas sobre las que no sabemos su comportamiento.
- Another Test: Añade un nuevo test para cubrir un caso adicional o una variante del comportamiento.
- Regression Test: Agrega un test que reproduce un bug descubierto, para evitar regresiones futuras.

estrategias para llegar al verde lo mas rápido posible

Patrones *green bar*

- Fake it: Devuelve un valor fijo o solución "falsa" solo para hacer pasar el test.
- Triangulación: Introduces un **segundo test con diferentes datos** para forzar una implementación más general
- Obvious implementation: Cuando la solución es trivial y obvia, puedes implementarla directamente sin "Fake It".
- One to many: Patrón para resolver test para colecciones de elementos.

¿Cómo hacer Refactoring?



15

En resumen

